

Raspberry Pi TeXNote 版組サーバー構築手順

状態確認から pdfLaTeX/CJK 用 Type 1 フォント導入まで

2026 年 8 月 15 日

概要

iPad 版・iPhone 版 TeXNote から、同一 LAN 上の Raspberry Pi へ TeX 文書を送り、PDF を生成するサーバーを再構築するための手順書である。Debian 12、TeX Live、FastAPI/Uvicorn、API トークン、systemd による自動起動、および pdfLaTeX/CJK で必要になる Type 1 フォント一式までを扱う。

目次

1	完成時の構成	3
2	Raspberry Pi の状態確認	3
3	TeX Live と日本語環境	4
4	5 種類のエンジンを単体テスト	4
5	Python 版組 API	5
5.1	仮想環境	5
5.2	Mac からプログラムを転送	6
6	API トークン	6
7	手動起動と疎通確認	7
8	API リクエストと添付ファイルの制限	7
9	systemd による自動起動	8
10	iPad 版・iPhone 版の設定	9
11	プログラムの更新	9
11.1	systemd 版の更新	9
12	画像・ファイル・独自スタイル	10
13	pdfLaTeX/CJK 用 Type 1 フォント一式	10
13.1	症状と事前確認	10
13.2	導入	10
14	日常の確認	11
15	完了チェックリスト	11

16	Docker 版への移行	12
16.1	移行方針と注意事項	12
16.2	Docker Engine と Compose の導入	12
16.3	Docker 用ディレクトリの準備	13
16.4	Dockerfile の作成	13
16.5	ビルドコンテキストから秘密情報を除外	14
16.6	Compose 設定の作成	15
16.7	イメージのビルドと TeX 環境の検査	16
16.8	systemd 版から Docker 版への切替	16
16.9	再起動後の自動復旧試験	17
16.10	更新、ログ、停止	17
16.11	問題発生時に systemd 版へ戻す	18
16.12	Docker 版完了チェックリスト	19
16.13	Docker 公式資料	19

1 完成時の構成

実際に構築した環境は次のとおりである。再構築時に異なる値は読み替える。

項目	値
機種	Raspberry Pi 5 Model B Rev 1.0
OS	Debian GNU/Linux 12 (bookworm)
CPU・メモリ	aarch64・約 8 GiB
ストレージ	約 29 GB
ホスト名・ユーザー	raspberrypi・mat
LAN 内 IP アドレス	192.168.3.11
サーバーディレクトリ	/home/mat/texnote-server
待受ポート	TCP 8000
TeX エンジン	LuaLaTeX、XeLaTeX、pdfLaTeX、upLaTeX、pLaTeX

コマンドは「Raspberry Pi」または「Mac」と明記した側で実行する。

2 Raspberry Pi の状態確認

Mac から接続する。

```
ssh mat@192.168.3.11
```

Raspberry Pi 上で状態を確認する。

```
cat /etc/os-release
uname -m
df -h hostname -I
tr -d '\0' </proc/device-tree/model
echo
free -h
hostnamectl
timedatectl
sudo apt update
apt list --upgradable
test -f /var/run/reboot-required \
  && cat /var/run/reboot-required \
  || echo "再起動は要求されていません"
```

画像ファイルが eps であった場合、gs (ghostscript) が必要になる

```
sudo apt install ghostscript
command -v gs
gs --version
```

概ね次の様に出れば問題なくインストールできている

```
/usr/bin/gs
10.00.0
```

必要なら更新して再起動する。

```
sudo apt full-upgrade
sudo reboot
```

IP が変わるとアプリから接続できなくなるため、ルーターの DHCP 予約を推奨する。

3 TeX Live と日本語環境

まず導入をシミュレーションする。

```
sudo apt-get --simulate install \
  texlive \
  texlive-latex-extra \
  texlive-luatex \
  texlive-xetex \
  texlive-lang-japanese \
  texlive-pictures \
  texlive-science \
  fonts-noto-cjk
```

大量の削除がないことを確認後、実際に導入する。

```
sudo apt install \
  texlive \
  texlive-latex-extra \
  texlive-luatex \
  texlive-xetex \
  texlive-lang-japanese \
  texlive-pictures \
  texlive-science \
  fonts-noto-cjk
```

エンジンを確認する。

```
command -v lualatex
command -v xelatex
command -v pdflatex
command -v uplatex
command -v platex
command -v dvipdfmx
command -v extractbb
lualatex --version | head -n 1
xelatex --version | head -n 1
pdflatex --version | head -n 1
uplatex --version | head -n 1
platex --version | head -n 1
dvipdfmx --version | head -n 1
```

4 5 種類のエンジンを単体テスト

```
mkdir -p ~/tex-tests
cd ~/tex-tests
```

```

cat > lualatex-test.tex <<'EOF'
\documentclass{ltjsarticle}
\begin{document}
LuaLaTeX 日本語テスト。 \quad  $e^{i\pi}+1=0$ 
\end{document}
EOF
lualatex -interaction=nonstopmode -halt-on-error lualatex-test.tex

cat > xelatex-test.tex <<'EOF'
\documentclass[xelatex,ja=standard]{bxjsarticle}
\begin{document}
XeLaTeX 日本語テスト。 \quad  $e^{i\pi}+1=0$ 
\end{document}
EOF
xelatex -interaction=nonstopmode -halt-on-error xelatex-test.tex

cat > pdflatex-test.tex <<'EOF'
\documentclass{article}
\begin{document}
pdfLaTeX test. \quad  $e^{i\pi}+1=0$ 
\end{document}
EOF
pdflatex -interaction=nonstopmode -halt-on-error pdflatex-test.tex

cat > uplateg-test.tex <<'EOF'
\documentclass[uplateg,dvipdfmx]{jsarticle}
\begin{document}
upLaTeX 日本語テスト。 \quad  $e^{i\pi}+1=0$ 
\end{document}
EOF
uplateg -interaction=nonstopmode -halt-on-error uplateg-test.tex
dvipdfmx uplateg-test.dvi

cat > platex-test.tex <<'EOF'
\documentclass[dvipdfmx]{jsarticle}
\begin{document}
pLaTeX 日本語テスト。 \quad  $e^{i\pi}+1=0$ 
\end{document}
EOF
platex -interaction=nonstopmode -halt-on-error platex-test.tex
dvipdfmx platex-test.dvi

ls -lh *-test.pdf
file *-test.pdf

```

pdfLaTeX はまず欧文と数式で確認し、CJK 日本語用フォントは後節で導入する。

5 Python 版組 API

5.1 仮想環境

Raspberry Pi 上で実行する。

```
sudo apt install python3-venv python3-pip openssl
mkdir -p /home/mat/texnote-server
cd /home/mat/texnote-server
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install 'fastapi==0.141.1' 'uvicorn[standard]==0.52.1'
python --version
python -m pip show fastapi | grep Version
python -m pip show uvicorn | grep Version
which python
which uvicorn
deactivate
```

再現用 requirements.txt は次の内容である。

```
fastapi==0.141.1
uvicorn[standard]==0.52.1
```

5.2 Mac からプログラムを転送

Mac 側のリポジトリ位置が異なる場合はパスを変更する。

```
scp \
/Users/mat/Documents/Git/TeXNote/Server/P2/progs/app.py \
/Users/mat/Documents/Git/TeXNote/Server/P2/progs/requirements.txt \
mat@192.168.3.11:/home/mat/texnote-server/
```

Raspberry Pi で依存関係をそろえる。

```
cd /home/mat/texnote-server
source .venv/bin/activate
python -m pip install -r requirements.txt
```

6 API トークン

Raspberry Pi 上で作成する。トークンは Git へ登録しない。

```
cd /home/mat/texnote-server
umask 077
openssl rand -hex 32 > .api-token
chmod 600 .api-token
printf 'TEXNOTE_API_TOKEN=' > server.env
cat .api-token >> server.env
chmod 600 server.env
ls -l .api-token server.env
```

7 手動起動と疎通確認

Raspberry Pi で起動する。

```
cd /home/mat/texnote-server
source .venv/bin/activate
set -a
source server.env
set +a
uvicorn app:app --host 192.168.3.11 --port 8000
```

Mac の別ターミナルで確認する。

```
curl --fail-with-body http://192.168.3.11:8000/health
```

期待される応答例：

```
{"status": "ok", "engines": ["lualatex", "xelatex", "pdflatex", "uplatex", "platex"]}
```

認証も確認し、Mac 上の一時変数を最後に消去する。

```
TEXNOTE_TOKEN="$(ssh mat@192.168.3.11 \
  'cat /home/mat/texnote-server/.api-token')"
```

```
curl --fail-with-body \
  http://192.168.3.11:8000/v1/auth-check \
  -H "Authorization: Bearer $TEXNOTE_TOKEN"
```

```
unset TEXNOTE_TOKEN
```

成功時は {"status": "ok"} が返る。手動サーバーは Pi 側で Ctrl-C を押して終了する。

8 API リクエストと添付ファイルの制限

P2 は過大な入力によるメモリ・CPU・一時領域の消費を抑えるため、現行の app.py で次の上限を設定している。

対象	現行上限	超過時の扱い
HTTP リクエスト全体	32 MiB	HTTP 413
TeX ソース	2,000,000 文字	入力検証エラー
添付 1 ファイル	10 MiB	HTTP 413
添付ファイル合計	20 MiB	HTTP 413
pictures の個数	50 個	入力検証エラー
files の個数	50 個	入力検証エラー
相対パス	1,024 文字	入力検証エラー
P2 ログ	200,000 文字	末尾を応答に使用

HTTP リクエスト全体の上限は次の定数で 33,554,432 bytes に設定する。

```
MAX_REQUEST_BYTES = 32 * 1024 * 1024
```

画像と添付ファイルは JSON へ Base64 文字列として格納される。Base64 は元データのおよそ 4/3 倍になるた

め、復号後 20 MiB の添付は約 26.7 MiB になる。これに TeX ソース、相対パス、JSON 構造を加えても収まるよう、HTTP 全体を 32 MiB としている。

Content-Length が 32 MiB を超える場合は、本文を解析する前に HTTP 413 を返す。

```
{"message": "リクエストの上限 32 MiB を超えています。"}

```

P1 にも同じ 32 MiB 制限を設定し、P1 で受理された要求が P2 の入口で容量超過にならないよう上限を統一する。公開時にリバースプロキシを置く場合も、そこで同じ上限を設定する。

9 systemd による自動起動

Raspberry Pi 上で作成する。

```
sudo nano /etc/systemd/system/texnote-server.service

```

内容は次のとおり。ExecStart は 1 行で記述する。

```
[Unit]
Description=TeXNote Typesetting Server
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=mat
Group=mat
WorkingDirectory=/home/mat/texnote-server
EnvironmentFile=/home/mat/texnote-server/server.env
ExecStart=/home/mat/texnote-server/.venv/bin/uvicorn app:app --host 192.168.3.11 --port 8000
Restart=on-failure
RestartSec=3
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target

```

読み込み、自動起動を有効化する。

```
sudo systemctl daemon-reload
sudo systemctl enable --now texnote-server
systemctl status texnote-server --no-pager
systemctl is-enabled texnote-server
systemctl is-active texnote-server
journalctl -u texnote-server -n 100 --no-pager

```

再起動試験をする。

```
sudo reboot

```

Pi が戻った後、Mac で確認する。

```
ping -c 3 192.168.3.11

```



```
curl --connect-timeout 5 http://192.168.3.11:8000/health
ssh mat@192.168.3.11 \
  'systemctl is-enabled texnote-server && systemctl is-active texnote-server'
```

10 iPad 版・iPhone 版の設定

アプリへ次を登録する。

- サーバー URL : `http://192.168.3.11:8000`
- API トークン : Pi の `/home/mat/texnote-server/.api-token` の内容

現段階は同一 Wi-Fi 内だけで使用する。ルーターの 8000 番ポートをインターネットへ転送しない。外出先利用は Tailscale や WireGuard 等の VPN を導入してから行う。

11 プログラムの更新

11.1 systemd 版の更新

アプリと版組サーバーの API を変更した場合は、両者の対応版を同時期に更新する。特に添付リソースのキーを `fileName` から `relativePath` へ変更した版では、新旧のアプリとサーバーを混在させると版組要求が HTTP 422 で失敗する。

まず Mac から新しい `app.py` を一時名で転送する。

```
scp \
  /Users/mat/Documents/Git/TeXNote/Server/P2/progs/app.py \
  mat@192.168.3.11:/home/mat/texnote-server/app.py.new
```

Pi 上で現行版をバックアップし、差替え前に Python 構文検査を行う。

```
ssh mat@192.168.3.11
cd /home/mat/texnote-server
cp -p app.py app.py.backup
.venv/bin/python -m py_compile app.py.new
mv app.py.new app.py
```

サービスを再起動し、状態、ログ、HTTP 応答を確認する。

```
sudo systemctl restart texnote-server
sudo systemctl status texnote-server --no-pager
sudo journalctl -u texnote-server -n 100 --no-pager
curl --fail-with-body http://192.168.3.11:8000/health
```

異常があればバックアップへ戻す。

```
cd /home/mat/texnote-server
cp -p app.py.backup app.py
sudo systemctl restart texnote-server
sudo systemctl status texnote-server --no-pager
```

サーバーの `/health` 確認後に対応する iPad・iPhone 版アプリを更新し、サブフォルダ内の画像または添付ファイルを含む Card で版組、保存、再読込を確認する。

12 画像・ファイル・独自スタイル

文書固有の.sty、画像、データは TeXNote の「画像・ファイル」から送る。新しい API では各リソースがノートルート基準の安全な相対パスを持ち、サーバー上の一時作業ディレクトリにもその階層が再現される。例えば files/以下のリソースは次のように参照する。

```
\usepackage{files/my-style}
\includegraphics{files/example.png}
\input{files/part.tex}
```

全文書で共通利用するスタイルを Pi のローカル TeX ツリーへ置く例：

```
mkdir -p ~/texmf/tex/latex/texnote
cp my-style.sty ~/texmf/tex/latex/texnote/
mktexlsr ~/texmf
kpsewhich my-style.sty
```

13 pdfLaTeX/CJK 用 Type 1 フォント一式

13.1 症状と事前確認

次のようなエラーは、IPAex の TrueType フォントだけでなく、pdfLaTeX/CJK が使う TFM/PFB/map が不足していることを示す。

```
I can't find file 'ipxm-r-u61'
Font ... =ipxm-r-u61 ... not loadable:
Metric (TFM) file not found.
Fatal error occurred, no output PDF file produced!
```

```
kpsewhich ipaexm.ttf
kpsewhich ipaexg.ttf
kpsewhich ipxm-r-u61.tfm
```

13.2 導入

IPAex フォントを確認・導入する。

```
sudo apt update
sudo apt install fonts-ipaexfont-mincho fonts-ipaexfont-gothic
```

Debian では Type 1 フォント一式を texlive-fonts-extra で導入する。容量が大きいため、先に空き容量を確認する。

```
df -h /
sudo apt install texlive-fonts-extra
sudo mktexlsr
sudo updmap-sys
```

導入結果を確認する。

```
kpsewhich ipaexm.ttf
kpsewhich ipaexg.ttf
kpsewhich ipxm-r-u61.tfm
kpsewhich ipxm-r-u61.pfb
kpsewhich ipaex-type1.map
```

各コマンドでパスが表示されれば TeX Live から発見できている。

```
sudo systemctl restart texnote-server
systemctl is-active texnote-server
```

14 日常の確認

```
systemctl status texnote-server --no-pager
journalctl -u texnote-server -n 100 --no-pager
journalctl -u texnote-server -f
curl --fail-with-body http://192.168.3.11:8000/health
df -h /
free -h
kpsewhich ファイル名.sty
```

15 完了チェックリスト

- ☐ OS、aarch64、容量、メモリ、IP を確認した。
- ☐ IP アドレスを DHCP 予約した。
- ☐ 5 種類の TeX エンジンが PDF を生成した。
- ☐ 仮想環境へ FastAPI と Uvicorn を導入した。
- ☐ API トークンの権限を 600 にした。
- ☐ /health と /v1/auth-check が成功した。
- ☐ systemd が enabled かつ active になった。
- ☐ Pi 再起動後も自動復旧した。
- ☐ iPad 版・iPhone 版の双方から版組できた。
- ☐ Type 1 の TFM/PFB/map を確認した。
- ☐ TCP 8000 をインターネットへ公開していない。

16 Docker 版への移行

ここまでは、Raspberry Pi 本体へ TeX Live と Python 仮想環境を導入し、systemd から Uvicorn を直接起動する構成を説明した。本節では、同じ版組 API と TeX 環境を Docker イメージへまとめる。

Docker 版はホスト側の Python 仮想環境や TeX Live に依存せず、Dockerfile、compose.yaml、app.py、requirements.txt から再構築できる。一方、TeX Live と texlive-fonts-extra を含むため、イメージは大きくなり、初回ビルドには長い時間と数 GB 以上の空き容量を要する。

16.1 移行方針と注意事項

本手順では次の構成にする。

- ベースイメージは arm64 対応の python:3.11-slim-bookworm を使う。
- TeX Live、日本語環境、IPAex、Type 1 フォントをイメージへ組み込む。
- API トークンはイメージへコピーせず、起動時に server.env から渡す。
- コンテナ内では root ではなく専用ユーザー texnote で実行する。
- コンテナは読み取り専用とし、版組用一時ファイルだけを /tmp へ置く。
- ホストの 192.168.3.11:8000 だけをコンテナへ転送する。
- 現在の systemd 版は、Docker 版のビルドと検査が終わるまで削除しない。

systemd 版と Docker 版は同じ TCP 8000 を同時に使用できない。Docker 版を起動する直前に systemd 版を停止し、問題があれば直ちに戻せるようにする。また、Docker が公開するポートは ufw の規則を迂回する場合がある。本例のように LAN 側 IP アドレスへ明示的に bind し、ルーターでは 8000 番を転送しない。

16.2 Docker Engine と Compose の導入

以下は Raspberry Pi 上で実行する。Docker 公式の Debian 用 APT リポジトリを使う。Debian 12 と arm64 は Docker Engine のサポート対象である。

既存の競合パッケージがないか確認する。

```
dpkg -l docker.io docker-compose docker-doc docker-buildx \
podman-docker containerd runc 2>/dev/null || true
```

既に別の Docker 環境を運用している場合は、ここで削除せず構成を確認する。未導入の環境では、公式鍵とリポジトリを登録する。

```
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg \
-o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: bookworm
Components: stable
Architectures: arm64
Signed-By: /etc/apt/keyrings/docker.asc
EOF
```

```
sudo apt update
sudo apt install \
  docker-ce \
  docker-ce-cli \
  containerd.io \
  docker-buildx-plugin \
  docker-compose-plugin
```

導入結果を確認する。

```
sudo systemctl is-enabled docker
sudo systemctl is-active docker
sudo docker version
sudo docker compose version
sudo docker run --rm hello-world
```

本書ではすべて `sudo docker` で実行する。ユーザーを `docker` グループへ追加すると `sudo` なしで操作できるが、そのユーザーには実質的に `root` 相当の権限が与えられるため、ここでは行わない。

16.3 Docker 用ディレクトリの準備

既存の `systemd` 版と混同しないよう、Pi 上に別ディレクトリを作る。

```
mkdir -p /home/mat/texnote-server-docker
chmod 700 /home/mat/texnote-server-docker
```

Mac 側から現在のサーバプログラムを転送する。

```
scp \
  /Users/mat/Documents/Git/TeXNote/Server/P2/progs/app.py \
  /Users/mat/Documents/Git/TeXNote/Server/P2/progs/requirements.txt \
  mat@192.168.3.11:/home/mat/texnote-server-docker/
```

API トークンは既存の値を引き継げる。Pi 上でコピーし、所有者以外から読めないようにする。

```
sudo install -o mat -g mat -m 600 \
  /home/mat/texnote-server/server.env \
  /home/mat/texnote-server-docker/server.env
```

新しいトークンへ交換する場合は次のように作る。この場合、iPad 版と iPhone 版の登録値も更新する。

```
cd /home/mat/texnote-server-docker
umask 077
printf 'TEXNOTE_API_TOKEN=' > server.env
openssl rand -hex 32 >> server.env
chmod 600 server.env
```

16.4 Dockerfile の作成

Pi 上の `/home/mat/texnote-server-docker/Dockerfile` を次の内容で作成する。

```
cd /home/mat/texnote-server-docker
nano Dockerfile
```

```
FROM python:3.11-slim-bookworm

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

RUN apt-get update \
    && DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends \
        texlive \
        texlive-latex-extra \
        texlive-luatex \
        texlive-xetex \
        texlive-lang-japanese \
        texlive-pictures \
        texlive-science \
        texlive-fonts-extra \
        fonts-noto-cjk \
        fonts-ipaexfont-mincho \
        fonts-ipaexfont-gothic \
    && mktexlsr \
    && updmap-sys \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt ./
RUN python -m pip install --no-cache-dir -r requirements.txt

COPY app.py ./

RUN useradd --system --uid 10001 --create-home texnote \
    && chown -R texnote:texnote /app

USER texnote

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health',
    ↪ timeout=4)" || exit 1

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

app.py は TeX エンジンを用usr/bin の絶対パスで呼び出す。Debian パッケージをコンテナへ導入しているため、既存コードを変更せず利用できる。

16.5 ビルドコンテキストから秘密情報を除外

/home/mat/texnote-server-docker/.dockerignore を作成する。

```
cd /home/mat/texnote-server-docker
nano .dockerignore
```

```
.git
.DS_Store
.venv
__pycache__
*.pyc
.api-token
server.env
docs
out
```

server.env を除外することが重要である。これにより、API トークンがイメージの layer や build context へ入らない。

16.6 Compose 設定の作成

/home/mat/texnote-server-docker/compose.yaml を作成する。

```
cd /home/mat/texnote-server-docker
nano compose.yaml
```

```
services:
  texnote-server:
    build:
      context: .
      dockerfile: Dockerfile
    image: texnote-server:bookworm
    restart: unless-stopped
    env_file:
      - ./server.env
    ports:
      - "192.168.3.11:8000:8000"
    read_only: true
    tmpfs:
      - /tmp:rw,noexec,nosuid,size=2g,mode=1777
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
    pids_limit: 128
```

この API は版組ごとに/tmp 下へ一時ディレクトリを作り、処理後に削除する。したがって永続 volume は不要である。read_only と tmpfs により、イメージ本体は変更せず一時領域だけを書き込み可能にする。

設定と秘密情報の権限を確認する。

```
cd /home/mat/texnote-server-docker
chmod 600 server.env
ls -la
sudo docker compose config --quiet
```

`docker compose config` を引数なしで実行すると、展開後の環境変数が表示される場合がある。トークンを画面やログへ出したいくない場合は `--quiet` だけを使う。

16.7 イメージのビルドと TeX 環境の検査

`systemd` 版を動かしたままイメージをビルドできる。初回は TeX Live 一式をダウンロードするため時間がかかる。

```
cd /home/mat/texnote-server-docker
df -h /
sudo docker compose build
sudo docker image ls texnote-server:bookworm
sudo docker system df
```

API サーバーを公開する前に、完成したイメージ内を検査する。

```
sudo docker compose run --rm --no-deps texnote-server \
  sh -lc 'uname -m && python --version'

sudo docker compose run --rm --no-deps texnote-server \
  sh -lc 'command -v lualatex; command -v xelatex; command -v pdflatex; command -v uplatex;
↪ command -v platex; command -v dvipdfmx; command -v extractbb'

sudo docker compose run --rm --no-deps texnote-server \
  sh -lc 'kpsewhich ipaexm.ttf; kpsewhich ipaexg.ttf; kpsewhich ipxm-r-u61.tfm; kpsewhich
↪ ipxm-r-u61.pfb; kpsewhich ipaex-type1.map'
```

すべての実行ファイルとフォントでパスが表示されることを確認する。

16.8 systemd 版から Docker 版への切替

まず `systemd` 版の現在状態を記録する。

```
systemctl is-enabled texnote-server
systemctl is-active texnote-server
```

Docker 版とポートが競合しないよう、`systemd` 版を停止して自動起動を無効にする。サービスファイルや Python 仮想環境は、切替確認が終わるまで削除しない。

```
sudo systemctl disable --now texnote-server
systemctl is-active texnote-server
```

Docker 版をバックグラウンド起動する。

```
cd /home/mat/texnote-server-docker
sudo docker compose up -d
sudo docker compose ps
sudo docker compose logs --tail=100 texnote-server
```

Pi 上で確認する。

```
curl --fail-with-body http://127.0.0.1:8000/health
sudo docker inspect \
```



```
--format '{{.State.Status}}' {{if .State.Health}}{{.State.Health.Status}}{{end}}' \
texnote-server-docker-texnote-server-1
```

Compose の project 名やディレクトリ名を変えた場合、コンテナ名も変わる。正確な名前は `sudo docker compose ps` で確認する。

Mac から LAN 経由で確認する。

```
curl --fail-with-body http://192.168.3.11:8000/health

TEXNOTE_TOKEN="$(ssh mat@192.168.3.11 \
  "sed -n 's/^TEXNOTE_API_TOKEN=//p' /home/mat/texnote-server-docker/server.env")"
curl --fail-with-body \
  http://192.168.3.11:8000/v1/auth-check \
  -H "Authorization: Bearer $TEXNOTE_TOKEN"
unset TEXNOTE_TOKEN
```

成功時は `{"status":"ok"}` が返る。続いて iPad 版・iPhone 版から実際に 5 種類のエンジンを順に版組する。

16.9 再起動後の自動復旧試験

Compose の restart: unless-stopped と Docker サービスの自動起動を確認する。

```
sudo reboot
```

Pi が戻った後、Mac から確認する。

```
ping -c 3 192.168.3.11
curl --connect-timeout 10 --fail-with-body \
  http://192.168.3.11:8000/health
ssh mat@192.168.3.11 \
  'sudo systemctl is-active docker; cd /home/mat/texnote-server-docker && sudo docker compose ps'
```

16.10 更新、ログ、停止

16.10.1 Docker Compose 版の更新

app.py または requirements.txt を更新した場合は、まず Pi 上の現行ファイルをバックアップする。

```
ssh mat@192.168.3.11
cd /home/mat/texnote-server-docker
cp -p app.py app.py.backup
cp -p requirements.txt requirements.txt.backup
exit
```

Mac から更新ファイルを転送する。

```
scp \
  /Users/mat/Documents/Git/TeXNote/Server/P2/progs/app.py \
  /Users/mat/Documents/Git/TeXNote/Server/P2/progs/requirements.txt \
  mat@192.168.3.11:/home/mat/texnote-server-docker/
```

Pi 上で Python 構文と Compose 設定を検査してから、イメージを再ビルドする。単なる `docker compose`

restart ではイメージ内の app.py が更新されないため、必ず--build を付ける。

```
ssh mat@192.168.3.11
cd /home/mat/texnote-server-docker
python3 -m py_compile app.py
sudo docker compose config --quiet
sudo docker compose up -d --build
sudo docker compose ps
sudo docker compose logs --tail=100 texnote-server
curl --fail-with-body http://192.168.3.11:8000/health
```

異常があればバックアップを戻し、旧イメージを再ビルドする。

```
cd /home/mat/texnote-server-docker
cp -p app.py.backup app.py
cp -p requirements.txt.backup requirements.txt
sudo docker compose up -d --build
sudo docker compose ps
```

添付リソース API を fileName から relativePath へ変更した版では、Docker 版の/health 確認後に対応する iPad・iPhone 版アプリを更新する。更新後はサブフォルダ内のリソースを含む Card で版組、保存、再読込を確認する。

日常の確認には次を使う。

```
cd /home/mat/texnote-server-docker
sudo docker compose ps
sudo docker compose logs --tail=100 texnote-server
sudo docker compose logs -f texnote-server
sudo docker stats --no-stream
sudo docker system df
```

ログ追跡は Ctrl-C で終了してよい。コンテナ自体は停止しない。

明示的に停止・再開する場合：

```
cd /home/mat/texnote-server-docker
sudo docker compose stop
sudo docker compose start
```

コンテナと Compose 用 network を削除するが、ビルド済みイメージを残す場合：

```
cd /home/mat/texnote-server-docker
sudo docker compose down
```

未使用イメージだけを整理する場合は、削除対象を先に確認する。

```
sudo docker image ls
sudo docker image prune
```

docker system prune -a は広範囲のイメージや cache を削除するため、本手順では使用しない。

16.11 問題発生時に systemd 版へ戻す

Docker 版で問題が発生した場合は、次の順で元の構成へ戻す。

```
cd /home/mat/texnote-server-docker
sudo docker compose down
sudo systemctl enable --now texnote-server
systemctl is-enabled texnote-server
systemctl is-active texnote-server
curl --fail-with-body http://192.168.3.11:8000/health
```

Docker 版と systemd 版の両方を同時に起動しない。

16.12 Docker 版完了チェックリスト

- ☐ Docker Engine と Compose plugin が導入された。
- ☐ Docker サービスが enabled かつ active になった。
- ☐ `server.env` が build context から除外され、権限 600 になった。
- ☐ イメージ内で 5 種類の TeX エンジンを発見できた。
- ☐ IPAex と Type 1 の TFM/PFB/map を発見できた。
- ☐ systemd 版を停止してから Docker 版を起動した。
- ☐ `/health` と `/v1/auth-check` が成功した。
- ☐ iPad 版・iPhone 版から実際に版組できた。
- ☐ Pi 再起動後にコンテナが自動復旧した。
- ☐ TCP 8000 をインターネットへ公開していない。
- ☐ systemd 版への復旧手順を残している。

16.13 Docker 公式資料

- Debian への Docker Engine 導入：[Docker Docs: Install Docker Engine on Debian](#)
- Linux 版 Compose plugin：[Docker Docs: Install the Compose plugin](#)
- Compose の `env_file`：[Docker Docs: Set environment variables](#)
- Compose サービス設定：[Docker Docs: Define services](#)

補足

OS のメジャー更新、ユーザー名、IP アドレス、TeXNote の保存場所を変えた場合は、本文中のパスと設定値を一括して読み替えること。

ソース・プログラム

ソースコード 1 requirements.txt

```
1 fastapi==0.141.1
2 uvicorn[standard]==0.52.1
```

ソースコード 2 app.py

```
1 import asyncio
2 import base64
3 import binascii
4 import os
5 import resource
6 import shutil
7 import subprocess
```

```

8 import tempfile
9 from pathlib import Path
10 from typing import Literal
11 from uuid import UUID
12
13 from fastapi import Depends, FastAPI, Header, HTTPException, Request, status
14 from fastapi.responses import JSONResponse
15 from pydantic import BaseModel, ConfigDict, Field
16
17
18 MAX_REQUEST_BYTES = 32 * 1024 * 1024
19 MAX_SOURCE_CHARACTERS = 2_000_000
20 MAX_ASSET_BYTES = 10 * 1024 * 1024
21 MAX_TOTAL_ASSET_BYTES = 20 * 1024 * 1024
22 MAX_ASSETS_PER_FOLDER = 50
23 MAX_LOG_CHARACTERS = 200_000
24 PROCESS_TIMEOUT_SECONDS = 35
25
26 ENGINES: dict[str, tuple[str, bool]] = {
27     "lualatex": ("/usr/bin/lualatex", False),
28     "xelatex": ("/usr/bin/xelatex", False),
29     "pdflatex": ("/usr/bin/pdflatex", False),
30     "uplatex": ("/usr/bin/uplatex", True),
31     "platex": ("/usr/bin/platex", True),
32 }
33
34 compile_slots = asyncio.Semaphore(2)
35 app = FastAPI(title="TeXNote_Typesetting_Server", version="1.0.0")
36
37
38 class CardAsset(BaseModel):
39     model_config = ConfigDict(extra="forbid")
40
41     relativePath: str = Field(min_length=1, max_length=1024)
42     data: str = Field(min_length=1)
43
44
45 class TypesettingRequest(BaseModel):
46     model_config = ConfigDict(extra="forbid")
47
48     cardID: UUID
49     engine: Literal["lualatex", "xelatex", "pdflatex", "uplatex", "platex"]
50     source: str = Field(min_length=1, max_length=MAX_SOURCE_CHARACTERS)
51     pictures: list[CardAsset] = Field(max_length=MAX_ASSETS_PER_FOLDER)
52     files: list[CardAsset] = Field(max_length=MAX_ASSETS_PER_FOLDER)
53
54
55 class TypesettingResponse(BaseModel):
56     pdfBase64: str
57     log: str
58
59
60 class HealthResponse(BaseModel):
61     status: str
62     engines: list[str]
63
64
65 @app.middleware("http")
66 async def limit_request_size(request: Request, call_next):
67     content_length = request.headers.get("content-length")
68     if content_length is not None:
69         try:
70             if int(content_length) > MAX_REQUEST_BYTES:
71                 return JSONResponse(
72                     status_code=status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
73                     content={
74                         "message": "リクエストの上限32MiBを超えています。"
75                     },
76                 )
77         except ValueError:
78             return JSONResponse(
79                 status_code=status.HTTP_400_BAD_REQUEST,
80                 content={"message": "Content-Lengthが不正です。"},
81             )
82     return await call_next(request)
83
84
85 def require_api_token(authorization: str | None = Header(default=None)) -> None:
86     expected = os.environ.get("TEXNOTE_API_TOKEN")
87     if not expected:
88         raise HTTPException(
89             status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
90             detail="サーバーのAPIトークンが設定されていません。",
91         )
92     if authorization != f"Bearer {expected}":
93         raise HTTPException(
94             status_code=status.HTTP_401_UNAUTHORIZED,
95             detail="認証に失敗しました。",
96             headers={"WWW-Authenticate": "Bearer"},

```

```

97     )
98
99
100 @app.get("/health", response_model=HealthResponse)
101 async def health() -> HealthResponse:
102     available = [
103         engine
104         for engine, (executable, _) in ENGINES.items()
105         if os.access(executable, os.X_OK)
106     ]
107     return HealthResponse(status="ok", engines=available)
108
109
110 @app.get(
111     "/v1/auth-check",
112     dependencies=[Depends(require_api_token)],
113 )
114 async def auth_check() -> dict[str, str]:
115     return {"status": "ok"}
116
117
118 @app.post(
119     "/v1/typeset",
120     response_model=TypesettingResponse,
121     dependencies=[Depends(require_api_token)],
122 )
123 async def typeset(payload: TypesettingRequest) -> TypesettingResponse:
124     executable, produces_dvi = ENGINES[payload.engine]
125     if not os.access(executable, os.X_OK):
126         raise HTTPException(
127             status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
128             detail=f"TeXエンジンが見つかりません: {payload.engine}",
129         )
130
131     decoded_pictures, decoded_files = decode_assets(
132         payload.pictures,
133         payload.files,
134     )
135
136     async with compile_slots:
137         return await asyncio.to_thread(
138             compile_document,
139             executable,
140             produces_dvi,
141             payload.source,
142             decoded_pictures,
143             decoded_files,
144         )
145
146
147 def decode_assets(
148     pictures: list[CardAsset],
149     files: list[CardAsset],
150 ) -> tuple[list[tuple[str, bytes]], list[tuple[str, bytes]]]:
151     total_bytes = 0
152     seen_paths: set[str] = set()
153
154     def decode_folder(assets: list[CardAsset]) -> list[tuple[str, bytes]]:
155         nonlocal total_bytes
156         decoded: list[tuple[str, bytes]] = []
157
158         for asset in assets:
159             validate_relative_path(asset.relativePath)
160             if asset.relativePath in seen_paths:
161                 raise HTTPException(
162                     status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
163                     detail=f"相対パスが重複しています: {asset.relativePath}",
164                 )
165             seen_paths.add(asset.relativePath)
166
167             try:
168                 content = base64.b64decode(asset.data, validate=True)
169             except (binascii.Error, ValueError) as error:
170                 raise HTTPException(
171                     status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
172                     detail=f"Base64データが不正です: {asset.relativePath}",
173                 ) from error
174
175             if len(content) > MAX_ASSET_BYTES:
176                 raise HTTPException(
177                     status_code=status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
178                     detail=f"ファイルが大きすぎます: {asset.relativePath}",
179                 )
180             total_bytes += len(content)
181             if total_bytes > MAX_TOTAL_ASSET_BYTES:
182                 raise HTTPException(
183                     status_code=status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
184                     detail="添付ファイルの合計サイズが大きすぎます。",
185                 )

```

```

186         decoded.append((asset.relativePath, content))
187         return decoded
188
189     return decode_folder(pictures), decode_folder(files)
190
191
192 def validate_relative_path(relative_path: str) -> None:
193     path = Path(relative_path)
194     if (
195         path.is_absolute()
196         or relative_path.startswith("~")
197         or "\\\" in relative_path
198         or "\\x00\" in relative_path
199         or "/" in relative_path
200         or any(part in {"", ".", ".."} for part in path.parts)
201         or path.as_posix() != relative_path
202     ):
203         raise HTTPException(
204             status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
205             detail=f"利用できない相対パスです: {relative_path}",
206         )
207
208
209 def compile_document(
210     executable: str,
211     produces_dvi: bool,
212     source: str,
213     pictures: list[tuple[str, bytes]],
214     files: list[tuple[str, bytes]],
215 ) -> TypesettingResponse:
216     with tempfile.TemporaryDirectory(prefix="texnote-") as temporary_path:
217         work_directory = Path(temporary_path)
218         write_inputs(work_directory, source, pictures, files)
219         environment = restricted_environment(work_directory)
220         log_parts: list[str] = []
221
222         if produces_dvi:
223             for relative_path, _ in pictures:
224                 if Path(relative_path).suffix.lower() in {".jpg", ".jpeg", ".png", ".pdf"}:
225                     log_parts.append(
226                         run_process(
227                             ["/usr/bin/extractbb", "-x", relative_path],
228                             work_directory,
229                             environment,
230                         )
231                     )
232
233         tex_arguments = [
234             executable,
235             "-no-shell-escape",
236             "-interaction=nonstopmode",
237             "-file-line-error",
238             "-halt-on-error",
239             "main.tex",
240         ]
241         for _ in range(2):
242             log_parts.append(
243                 run_process(tex_arguments, work_directory, environment)
244             )
245
246         if produces_dvi:
247             log_parts.append(
248                 run_process(
249                     ["/usr/bin/dvipdfmx", "main.dvi"],
250                     work_directory,
251                     environment,
252                 )
253             )
254
255         pdf_path = work_directory / "main.pdf"
256         if not pdf_path.is_file():
257             raise HTTPException(
258                 status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
259                 detail="PDFを生成できませんでした。",
260             )
261
262         log = "".join(log_parts)
263         return TypesettingResponse(
264             pdfBase64=base64.b64encode(pdf_path.read_bytes()).decode("ascii"),
265             log=log[-MAX_LOG_CHARACTERS:],
266         )
267
268
269 def write_inputs(
270     work_directory: Path,
271     source: str,
272     pictures: list[tuple[str, bytes]],
273     files: list[tuple[str, bytes]],
274 ) -> None:

```

```

275 (work_directory / "tex-cache").mkdir(mode=0o700)
276 (work_directory / "main.tex").write_text(source, encoding="utf-8")
277
278 for legacy_folder, assets in (("pics", pictures), ("files", files)):
279     for relative_path, content in assets:
280         destination = work_directory / relative_path
281         destination.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
282         destination.write_bytes(content)
283
284         legacy_destination = work_directory / legacy_folder / Path(relative_path).name
285         if legacy_destination != destination:
286             legacy_destination.parent.mkdir(
287                 mode=0o700,
288                 parents=True,
289                 exist_ok=True,
290             )
291             legacy_destination.write_bytes(content)
292
293
294 def restricted_environment(work_directory: Path) -> dict[str, str]:
295     return {
296         "HOME": str(work_directory),
297         "PATH": "/usr/bin:/bin",
298         "LANG": "C.UTF-8",
299         "LC_ALL": "C.UTF-8",
300         "openin_any": "p",
301         "openout_any": "p",
302         "TEXMFCACHE": str(work_directory / "tex-cache"),
303         "TEXMFVAR": str(work_directory / "tex-cache"),
304         "TEXMFOUTPUT": str(work_directory),
305     }
306
307
308 def apply_resource_limits() -> None:
309     resource.setrlimit(resource.RLIMIT_CPU, (30, 35))
310     resource.setrlimit(resource.RLIMIT_AS, (1_500_000_000, 1_500_000_000))
311     resource.setrlimit(resource.RLIMIT_FSIZE, (50_000_000, 50_000_000))
312     resource.setrlimit(resource.RLIMIT_NOFILE, (256, 256))
313
314
315 def run_process(
316     arguments: list[str],
317     work_directory: Path,
318     environment: dict[str, str],
319 ) -> str:
320     try:
321         result = subprocess.run(
322             arguments,
323             cwd=work_directory,
324             env=environment,
325             stdin=subprocess.DEVNULL,
326             stdout=subprocess.PIPE,
327             stderr=subprocess.STDOUT,
328             text=True,
329             errors="replace",
330             timeout=PROCESS_TIMEOUT_SECONDS,
331             check=False,
332             preexec_fn=apply_resource_limits,
333         )
334     except subprocess.TimeoutExpired as error:
335         raise HTTPException(
336             status_code=status.HTTP_408_REQUEST_TIMEOUT,
337             detail="版組処理がタイムアウトしました。",
338         ) from error
339
340     output = result.stdout[:-MAX_LOG_CHARACTERS:]
341     if result.returncode != 0:
342         raise HTTPException(
343             status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
344             detail=output or f"版組処理に失敗しました (終了値「{result.returncode}」)。",
345         )
346     return output
347
348
349 @app.exception_handler(HTTPException)
350 async def http_exception_handler(_request: Request, error: HTTPException):
351     return JSONResponse(
352         status_code=error.status_code,
353         content={"message": str(error.detail)},
354         headers=error.headers,
355     )

```